

Summary of Lectures

ITC2, Spring 2026

by:

Padmini Mukkamala

Budapest University of Technology and Economics

Last updated: March 7, 2026

The pseudocodes that appear here have been adapted from the Hungarian Course Notes.

Contents

1	Lecture 1: Introduction	2
1.1	Graphs: Basic Definitions	2
1.2	Isomorphism	8
1.3	Directed Graphs	10
2	Lecture 2	10
2.1	Walks, paths and cycles	10
2.2	Connected graphs	12
2.3	Trees	14
2.4	Forests	16
2.5	Spanning tree	16
2.6	Structural properties of trees	17
3	Lecture 3	18
3.1	Breadth First Search	18
3.2	Dijkstra's Algorithm	23
4	Lecture 4	25
4.1	The birth of graph theory	25
4.2	Eulerian circuits	26
4.3	Hamiltonian paths, cycles	28

1 Lecture 1: Introduction

Graphs, subgraph, complement, isomorphism, directed graphs.

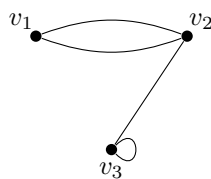
1.1 Graphs: Basic Definitions

Definition 1.1. A **graph** is a tuple $G = (V, E)$ where V (also written as $V(G)$) is the set of vertices and E (also written as $E(G)$) is a multiset of pairs of vertices.

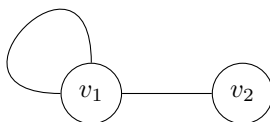
Definition 1.2. Two or more edges in the graph are said to be **multiple edges** (or **parallel edges**) if they connect the same pair of vertices. (Note, then this pair (u, v) occurs more than once in $E(G)$).

Definition 1.3. A pair of the form $(u, u) \in E(G)$ is called a **loop**. It is an edge that connects a vertex to itself.

Example 1.4. Let $G = (V, E)$ where $V = \{v_1, v_2, v_3\}$, $E = \{(v_1, v_2), (v_1, v_2), (v_2, v_3), (v_3, v_3)\}$. Then G has a multiple edge between v_1 and v_2 , and a loop at the vertex v_3 .

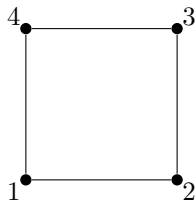


Example 1.5. Let $G = (V, E)$ where $V = \{v_1, v_2\}$, $E = \{(v_1, v_1), (v_1, v_2)\}$. The edge (v_1, v_1) is a loop at the vertex v_1 . Note that graphs can also be depicted as follows with the vertex labels in the center, but we will stick to the above way of depicting them.

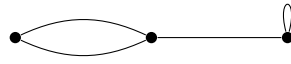


Definition 1.6. A graph is said to be a **simple graph** if it does not contain any multiple edges or loops.

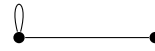
Example 1.7. Let $G = (V, E)$ where $V = \{1, 2, 3, 4\}$, $E = \{(1, 2), (2, 3), (3, 4), (4, 1)\}$. This graph has no loops and no multiple edges, and hence is a simple graph.



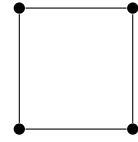
The following figures illustrate that graphs are often presented purely as drawings, without explicitly labeling vertices. The graphs presented below are the three graphs already discussed above.



Graph 1



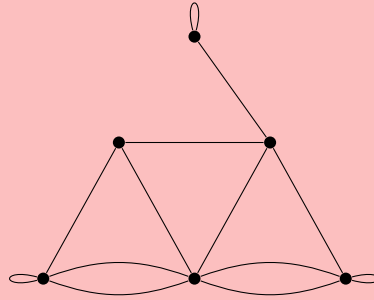
Graph 2



Graph 3

Example 1.8.

Exercise 1.1. Write the following graph as a tuple with a vertex and edge set.

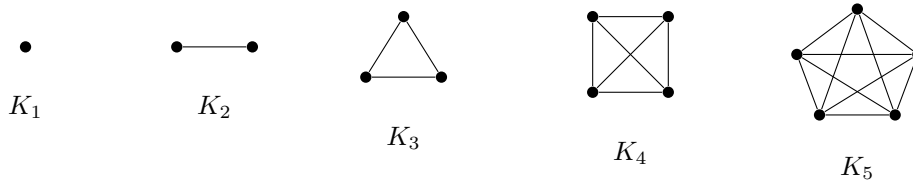


Exercise 1.2. Draw the graph $G = (V, E)$ where $V = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7\}$ and $E = \{(v_1, v_1), (v_1, v_2), (v_1, v_2), (v_2, v_3), (v_3, v_4), (v_4, v_4), (v_4, v_5), (v_5, v_6), (v_6, v_2), (v_7, v_7)\}$.

Exercise 1.3. Given a simple graph on n vertices, what is the maximum possible edges it can have?

Definition 1.9. A simple graph where $\forall u, v \in V(G)$ with $u \neq v$, $(u, v) \in E(G)$ is called a **complete graph**. A complete graph on n vertices is denoted by K_n .

Example 1.10. The first five complete graphs for $n \in \{1, 2, 3, 4, 5\}$ are given below.



Exercise 1.4. Let $f(n)$ denote the number of edges of K_n . Find a simple recurrence of $f(n)$ in terms of $f(n-1)$. Use this to reconfirm the result of the previous exercise.

There are two more special graphs we will discuss: Paths and Cycles.

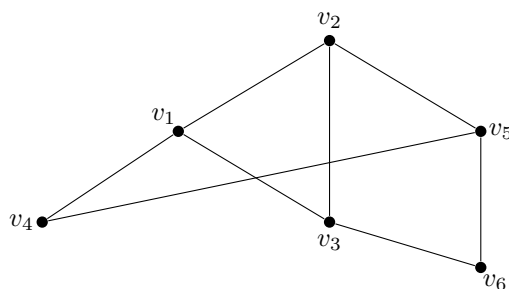
Definition 1.11. A path graph, denoted by P_n is a simple graph on n vertices $v_1, v_2, \dots, v_n$, such that its only edges are of the form (v_i, v_{i+1}) , where $1 \leq i < n$. We define the length of the path as the number of edges, and thereby it is $n-1$ for P_n .

Definition 1.12. A cycle graph, denoted by C_n is a simple graph on n vertices $v_1, v_2, \dots, v_n$, such that its edges are of the form (v_i, v_{i+1}) , where $1 \leq i < n$ along with the additional edge (v_n, v_1) . We define the length of the cycle as the number of edges, and thereby it is n for C_n .

Definition 1.13. The **degree** of a vertex v , denoted by $d(v)$, is the number of edges incident on it. Note: loops add two to the degree of the vertex they are incident on.

Definition 1.14. An **isolated vertex** is a vertex of a graph of degree 0. (Note: it is also called a trivial component, but this terminology will make more sense only after the second lecture).

Example 1.15. Consider the graph $G = (V, E)$ with $V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$



The degrees of the vertices are:

$$\begin{aligned} d(v_1) &= 3, & d(v_2) &= 3, & d(v_3) &= 3, \\ d(v_4) &= 2, & d(v_5) &= 3, & d(v_6) &= 2. \end{aligned}$$

Exercise 1.5. What are the degrees of the vertices of P_n ? What about C_n ? And lastly, what about K_n ?

Definition 1.16. The list of degrees of the vertices of the graph written in increasing order is said to be the **degree sequence** of the graph.

Example 1.17. Degree sequence of a graph.

For the graph G in the previous example, the list of vertex degrees is

$$\{3, 3, 3, 2, 3, 2\}.$$

Writing these degrees in increasing order, we obtain the degree sequence

$$(2, 2, 3, 3, 3, 3).$$

Example 1.18. Consider the graph $G = (V, E)$ with $V = \{v_1, v_2, v_3, v_4\}$ and edge multiset $E = \{(v_1, v_1), (v_1, v_2), (v_1, v_2), (v_2, v_3)\}$.



The degrees of the vertices are computed as follows:

$$\begin{aligned} d(v_1) &= 2 \text{ (from the loop)} + 2 \text{ (from the two edges to } v_2) = 4, \\ d(v_2) &= 2 \text{ (from the two edges to } v_1) + 1 \text{ (from the edge to } v_3) = 3, \\ d(v_3) &= 1, \\ d(v_4) &= 0. \end{aligned}$$

Thus, the degree sequence of G , since it is written in increasing order, is

$$(0, 1, 3, 4).$$

Further note that v_4 is an isolated vertex.

Exercise 1.6. Consider the degree sequence $(1, 1, 1, 1, 1)$. Is there a graph with such a degree sequence?

Theorem 1.19. (Handshake theorem): For any graph G , $\sum_v d(v) = 2|E(G)|$. Note: the graph does not need to be simple.

Proof. When counting the sum of degrees, every edge is counted precisely twice, once for each vertex incident on it. Loops add 2 to the degree of the vertices by definition. $\square$

Exercise 1.7. Consider the degree sequence $(4, 5, 6, 7, 8)$. Is there a graph with such a degree sequence?

Exercise 1.8. Show that for any degree sequence whose sum is even, you can always find a graph for it.

Exercise 1.9. Consider the degree sequence $(4, 5, 6, 7, 8)$. Is there a simple graph with such a degree sequence?

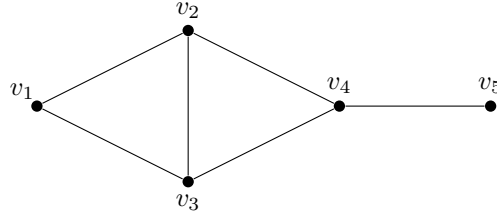
Exercise 1.10. Consider the degree sequence $(0, 1, 2, 3, 4)$. Is there a simple graph with such a degree sequence?

Sequences which can be the degree sequence of a simple graph are completely characterized. It is not in our syllabus, but interested readers are urged to think about it.

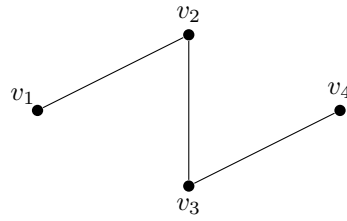
Definition 1.20. A graph $G' = (V', E')$ is said to be a **subgraph** of $G = (V, E)$, if $V' \subseteq V$ and $E' \subseteq E$.

Definition 1.21. A graph $G' = (V', E')$ is said to be an **induced subgraph** of $G = (V, E)$, if $V' \subseteq V$ and $E' \subseteq E$, and $\forall u, v \in V'$, $(u, v) \in E'$ if and only if $(u, v) \in E$. For a subset $U \subseteq V$, we denote the induced subgraph of G on the vertices of U by $G[U]$.

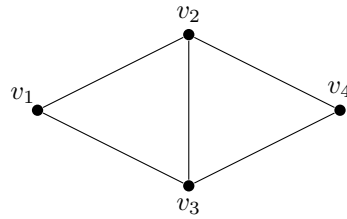
Example 1.22. Let $G = (V, E)$ be the graph in the picture and let $U = \{v_1, v_2, v_3, v_4\}$.



A subgraph. Let H be the graph with vertex set U and edge set $E(H) = \{(v_1, v_2), (v_2, v_3), (v_3, v_4)\}$. Then H is a subgraph of G , since its vertices and edges are contained in G .



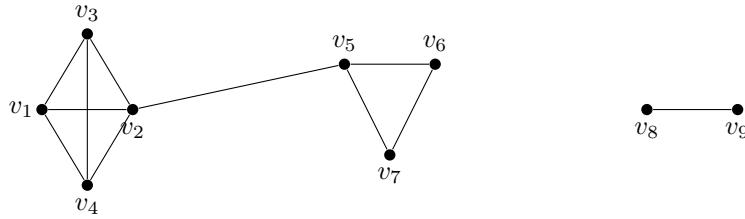
The induced subgraph on U . The induced subgraph $G[U]$ has vertex set U and contains all edges of G with both endpoints in U , namely, $E(G[U]) = \{(v_1, v_2), (v_1, v_3), (v_2, v_3), (v_2, v_4), (v_3, v_4)\}$.



Thus, every induced subgraph is a subgraph, but not every subgraph is an induced subgraph.

Definition 1.23. A **clique** in a simple graph G is a complete subgraph of G . The size of the clique is the number of vertices in it. Any non-empty graph has a clique of size 1, while a simple graph with at least one edge has a clique of size 2.

Example 1.24. Consider the simple graph $G = (V, E)$ shown below.

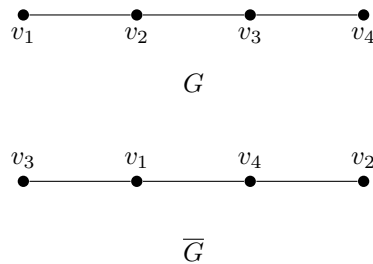


This graph contains several cliques of different sizes. For example, $\{v_1, v_2, v_3, v_4\}$ forms a clique of size 4, $\{v_5, v_6, v_7\}$ and any three vertices out of $\{v_1, v_2, v_3, v_4\}$ form a clique of size 3, the endpoints of every edge form a clique of size 2. Note: it is incorrect to think that only $\{v_8, v_9\}$ forms a clique of size 2, a clique is a subgraph but it need not be isolated or separate from the graph in any sense. Each individual vertex also forms a clique of size 1, in accordance with the definition.

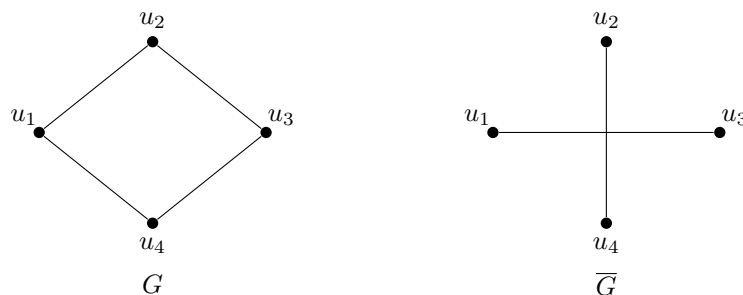
Exercise 1.11. A simple graph on 7 vertices has C_6 as an induced subgraph. How many edges can it have?

Definition 1.25. The **complement** of a simple graph $G = (V, E)$, denoted by $\overline{G}$, is a simple graph such that $V(\overline{G}) = V$ and $(u, v) \in E(\overline{G})$ if and only if $(u, v) \notin E$. So, the complement is the graph with the same vertex set V such that two distinct vertices are adjacent in $\overline{G}$ if and only if they are not adjacent in G .

Example 1.26. (a) Let $V = \{v_1, v_2, v_3, v_4\}$, $E(G) = \{(v_1, v_2), (v_2, v_3), (v_3, v_4)\}$.



(b) Let $V = \{u_1, u_2, u_3, u_4\}$, $E(G) = \{(u_1, u_2), (u_2, u_3), (u_3, u_4), (u_4, u_1)\}$.



Exercise 1.12. Exercise: Show that $\overline{\overline{G}} = G$.

1.2 Isomorphism

When can we say that two simple graphs are the same?

Definition 1.27. Two simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are said to be **isomorphic** if there is a bijection (invertible function) $f : V_1 \rightarrow V_2$ such that $(u, v) \in E_1$ if and only if $(f(u), f(v)) \in E_2$. We denote this as $G_1 \cong G_2$.

Note: there are natural extensions to multigraphs, we won't discuss it in this class.

Exercise 1.13. Given simple graphs G_1 and G_2 such that $G_1 \cong G_2$, and let the bijection be given by the function f . Let $u \in V(G_1)$. Show that $d(u) = d(f(u))$.

The following three exercises will not be discussed in the lecture, but students are encouraged to think about them.

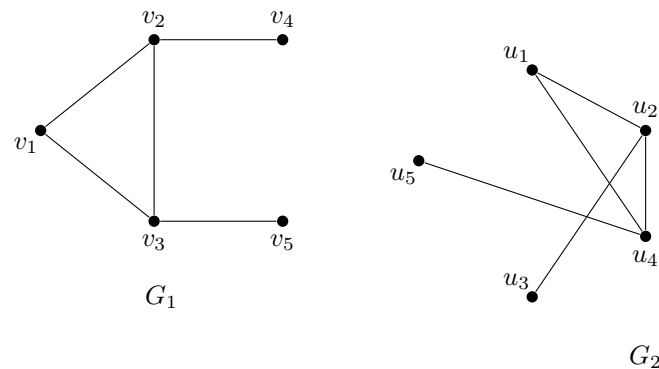
Exercise 1.14. Show that isomorphism preserves the degree sequence of a graph. That is, given graphs G_1 and G_2 such that $G_1 \cong G_2$, show that they have the same degree sequence.

Exercise 1.15. Show that isomorphism preserves paths and cycles of a graph. That is, given graphs G_1 and G_2 such that $G_1 \cong G_2$, show that if there is a path/cycle of length k in G_1 , then there is a path/cycle of length k in G_2 and vice versa.

Exercise 1.16. Show that isomorphism preserves subgraphs. That is, given graphs G_1 and G_2 such that $G_1 \cong G_2$, show that if H is a subgraph of G_1 , then H is also a subgraph of G_2 .

Example 1.28. (a) Two isomorphic graphs with different drawings.

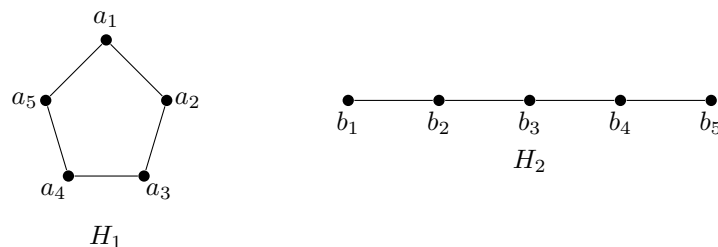
Let $V_1 = \{v_1, v_2, v_3, v_4, v_5\}$ and $E_1 = \{(v_1, v_2), (v_1, v_3), (v_2, v_3), (v_2, v_4), (v_3, v_5)\}$. Further, let $V_2 = \{u_1, u_2, u_3, u_4, u_5\}$ and $E_2 = \{(u_1, u_2), (u_1, u_4), (u_2, u_4), (u_4, u_5), (u_2, u_3)\}$.



Although the drawings of G_1 and G_2 look quite different, one can verify that there exists a bijection between their vertex sets preserving adjacency. Hence G_1 and G_2 are isomorphic.

(b) Two non-isomorphic graphs on five vertices.

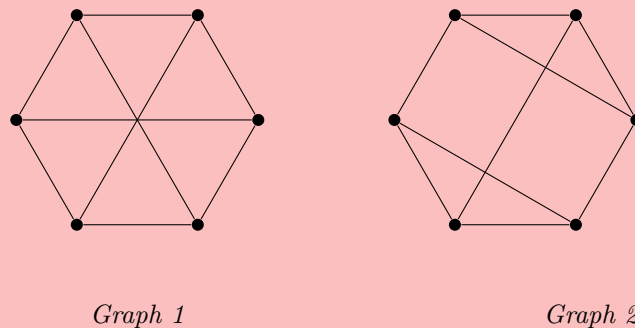
Let H_1 and H_2 be the two graphs given below:



The graphs H_1 and H_2 are not isomorphic, since H_1 has all vertices of degree 2, while H_2 has vertices of degree 1.

Exercise 1.17. Find the bijection between the vertices for the isomorphic graphs in Example (a) above.

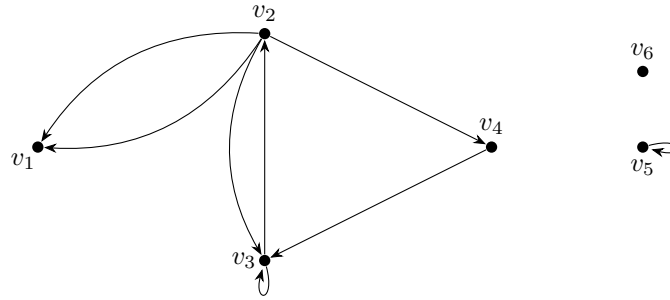
Exercise 1.18. Are the following two graphs isomorphic?



1.3 Directed Graphs

Definition 1.29. A **directed graph** is a tuple $G = (V, E)$ where V (also written as $V(G)$) is the set of vertices and E (also written as $E(G)$), is a multiset of **ordered pairs** of vertices.

Example 1.30. Consider the directed graph $G = (V, E)$ shown below.



In this directed graph:

- the edges (v_3, v_2) and (v_2, v_3) are **not** multiple edges,
- there are multiple directed edges from v_2 to v_1 ,
- vertices v_3 and v_5 have loops,
- the vertex v_5 is not an isolated vertex since it has a loop.
- the vertex v_6 is an isolated vertex.

Definition 1.31. The **indegree** of vertex v , denoted by $d^-(v)$, is the number of incoming edges incident on it, that is, edges of the form $\overrightarrow{uv}$. Note: loops only add one to the indegree.

The **outdegree** of vertex v , denoted by $d^+(v)$, is the number of outgoing edges incident on it, that is, edges of the form $\overrightarrow{vu}$. Note: loops only add one to the outdegree.

Exercise 1.19. For the example given above, determine the indegree and outdegree of every vertex.

Exercise 1.20. Given a directed graph G , what is $\sum_v d^-(v)$? And what is $\sum_v d^+(v)$?

2 Lecture 2

Connected graphs, Components, Trees, Spanning trees.

2.1 Walks, paths and cycles

Definition 2.1. We define the following ways of traversing the graph:

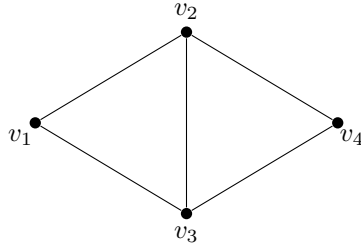
- **Walk:** A walk in a graph G is a sequence $u_0, e_1, u_1, e_2, u_2, \dots, e_k, u_k$ such that $e_i = (u_{i-1}, u_i) \in E(G)$. The length of the walk is k , the number of edges in it.
- **Trail:** A walk with no repeating edges, $e_i \neq e_j$ for $i \neq j$.
- **Closed walk:** A walk with $u_0 = u_k$.
- **Closed trail or circuit:** A trail with $u_0 = u_k$.

- **Path:** A walk with no repeating vertex, that is, $u_i \neq u_j$ for $i \neq j$.
- **Cycle:** Closed trail with no repeating vertex other than the first and the last. So $u_0 = u_k$ but for every other set of indices $i \neq j$, $u_i \neq u_j$.

Definition 2.2. The **length** of a walk/trail/path/cycle is the number of edges in it (counting multiplicities).

Note: In a **simple graph**, walks, trails and paths can instead just be denoted by the sequences of vertices, and we omit mentioning the edge between the consecutive vertices. We will use this shortened notation in the examples below.

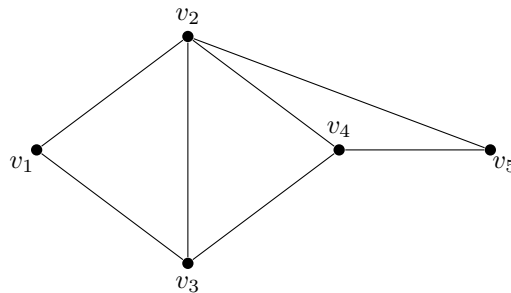
Example 2.3. Consider the graph $G = (V, E)$ shown below.



A walk. The sequence $v_1, (v_1, v_2), v_2, (v_2, v_3), v_3, (v_3, v_2), v_2, (v_2, v_4), v_4$, written in shortened form as v_1, v_2, v_3, v_2, v_4 is a walk of length 4. The edge (v_2, v_3) is repeated, so this walk is not a trail.

A trail with repeated vertices. The sequence v_2, v_1, v_3, v_2, v_4 is a trail of length 4. Although the vertex v_2 appears more than once, no edge is repeated.

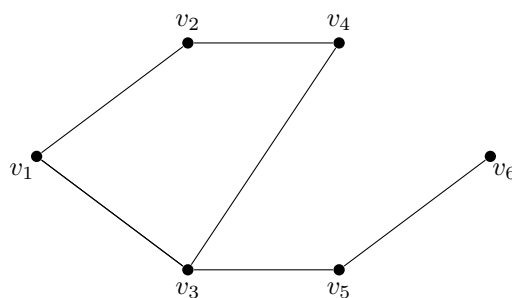
Example 2.4. Consider the graph $G = (V, E)$ shown below.



A closed walk. The sequence v_1, v_2, v_3, v_2, v_1 is a closed walk of length 4. The edge (v_2, v_3) is repeated, so this walk is not a closed trail.

A closed trail (circuit) with repeated vertices. The sequence $v_1, v_2, v_5, v_4, v_2, v_3, v_1$ is a closed trail of length 6. Although the vertices v_1 and v_2 appear more than once, no edge is repeated, and the walk starts and ends at the same vertex.

Example 2.5. Consider the graph $G = (V, E)$ shown below.



A path in G . The sequence v_2, v_4, v_3, v_5, v_6 is a path of length 4.

A cycle in G . The sequence v_1, v_2, v_4, v_3, v_1 is a cycle of length 4.

The notion of walks is remarkably useful in exploring the graphs and usually the first step of structural theorems. The following theorem and the exercises after illustrate this.

Theorem 2.6. Given a graph G with two vertices $u, v \in V(G)$, if there is a uv -walk in G , then there must be a uv -path in G .

Proof. Let $u, u_1, \dots, u_k, v$ be a uv -walk. If no vertex is repeated, then it is a path. If a vertex, say u_i is repeated, delete the walk between the first and last occurrence of u_i . Repeat until no vertex repeats. This process terminates because the number of edges in the walk is strictly decreasing with each iteration. $\square$

Exercise 2.1. Is it true that if a graph has a closed walk, then it must also contain a cycle? What if it has a closed trail (circuit)?

Exercise 2.2. Let G be a graph that is not empty (it contains at least one edge). Further assume that the degree of every vertex is even. Show that G contains a cycle.

Exercise 2.3. Given two vertices u, v in a graph G , if there is a path between u and v and a path between v and w , then show that there is also a path between u and w .

Definition 2.7. The **distance** between two vertices $u, v \in V(G)$, denoted by $d(u, v)$, is the length of the shortest uv -path. If no such path exists, we define $d(u, v) = \infty$.

Exercise 2.4. Show that the distance function satisfies triangle inequality, that is, $d(u, v) \leq d(u, w) + d(w, v)$ for any three vertices u, v and w .

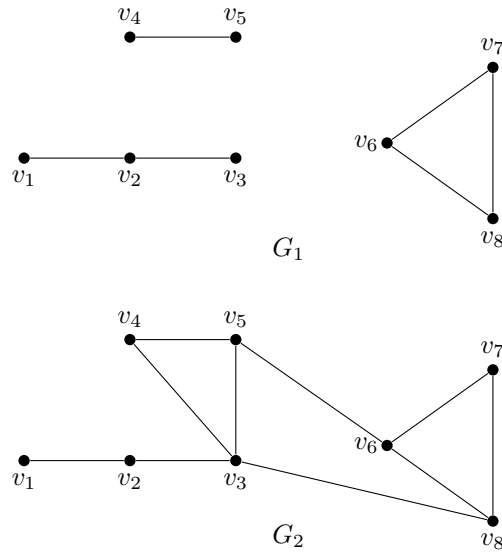
CAREFUL! The notation for degree is $d(u)$ while that for distance is $d(u, v)$.

2.2 Connected graphs

Definition 2.8. A graph is said to be **connected** if there is a path between every pair of vertices. Using the notion of distance, we can also say that a graph is **connected** if for every pair of vertices u, v , $d(u, v) < \infty$.

Definition 2.9. The **components** of a graph are its maximal connected subgraphs. That is, a component is a maximal subset of vertices where all pairwise distances are finite.

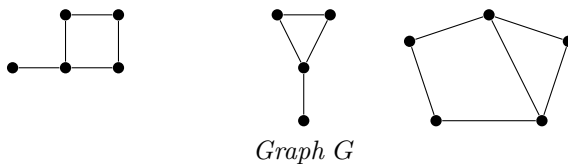
Example 2.10. Consider the following two graphs on the same vertex set $V = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8\}$.



The graph G_1 consists of three components. The graph G_2 is obtained from G_1 by adding edges so that every vertex can be reached from every other vertex.

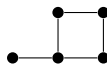
Exercise 2.5. In the above given example, what is the smallest possible number of edges one must add to G_1 to make it connected?

Example 2.11. Consider the graph G shown below. We will use it to illustrate the definition of components.



Graph G

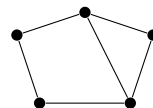
The Components of G : Each is a maximal connected subgraph.



Component 1



Component 2



Component 3

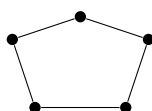
Why connectivity is not enough: A component must be **maximal**. Below are two examples of connected subgraphs that are not components because they can be extended within G .



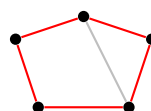
Connected subgraph



Not maximal (missing a vertex and an edge)



Connected subgraph

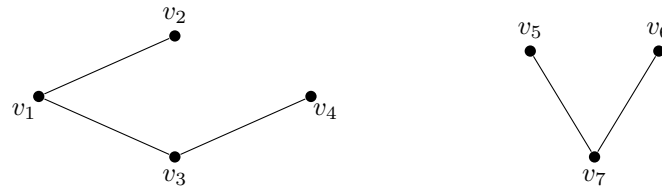


Not maximal (missing grey edge)

2.3 Trees

Definition 2.12. A graph is said to be **acyclic** if it does not contain any cycles.

Example 2.13. Consider the graph $G = (V, E)$ shown below.



This graph contains no cycles, and hence is acyclic.

Definition 2.14. A **Tree** is a connected acyclic graph.

Example 2.15. Below are all trees (up to isomorphism) on 1, 2, 3, and 4 vertices.

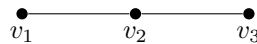
Trees on 1 vertex.



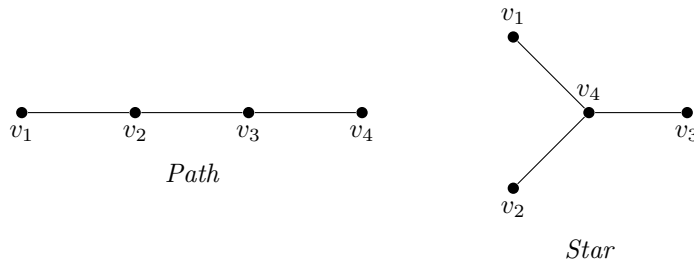
Trees on 2 vertices.



Trees on 3 vertices.



Trees on 4 vertices.



There is exactly one tree on 1, 2, and 3 vertices, and exactly two non-isomorphic trees on 4 vertices.

Exercise 2.6. List all the non-isomorphic trees on 5 and 6 vertices.

Definition 2.16. The degree one vertices of a tree are called **leaves**.

Theorem 2.17. Every tree with at least 2 vertices has at least 2 leaves.

Proof. Consider the longest path in G . The first and last vertices of the path must have degree 1 (leaves): if they had another neighbor not on the path, the path could be extended, and if the neighbor was on the path, it would form a cycle. $\square$

Theorem 2.18. *If v is a leaf of tree G , then $G \setminus v$ is also a tree. Note: $G \setminus v$ is the graph obtained by removing the vertex v and all edges incident on v .*

Proof. Deleting a leaf cannot disconnect the graph or induce cycles. $\square$

Theorem 2.19. *If G is a tree on n vertices, then it has $n - 1$ edges.*

Proof. We proceed by induction on n . The base case $n = 1$ is an isolated vertex, so the claim is true. For any larger $n > 1$, we delete a leaf of the tree. We observe that the new graph is a tree with $n - 1$ vertices, and so has $n - 2$ edges (induction hypothesis). Since G has just one additional leaf, the number of edges in G is $n - 1$. $\square$

Theorem 2.20. *Any connected graph G with $n - 1$ edges is a tree.*

Proof. If the graph G has a cycle, removing one edge from the cycle will not disconnect the graph. We repeat this process until G is acyclic. If G was not acyclic to start with, we will get a tree on n vertices with strictly fewer than $n - 1$ edges, a contradiction. $\square$

2.4 Forests

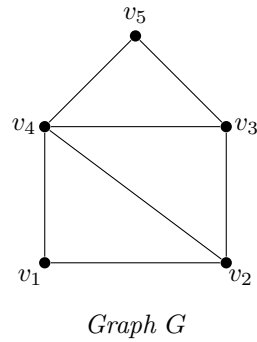
Definition 2.21. *A graph G is called a **forest** if all its components are trees.*

Exercise 2.7. *If a forest on n vertices has k components in it, how many edges does it have?*

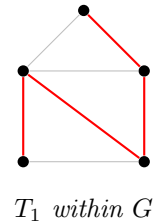
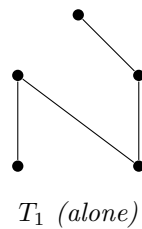
2.5 Spanning tree

Definition 2.22. *Given any graph G , a **spanning tree** T is a subgraph of G such that $V(T) = V(G)$ and T is a tree.*

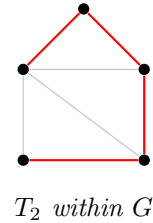
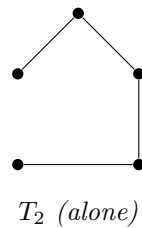
Example 2.23. Consider the connected graph G shown below. We can find multiple different spanning trees for this graph.



Spanning Tree T_1 :



Spanning Tree T_2 :



Theorem 2.24. A graph G has a spanning tree if and only if it is connected.

Proof. If a graph has a spanning tree, then it is clearly connected, because a tree is connected and the spanning tree is a subgraph of G . On the other hand, if G is connected, we repeat the algorithm of picking a cycle in G and removing an edge in it, so that we always get a subgraph of G that is still connected, but we keep reducing the number of cycles in it. This process terminates because the graph has finitely many edges, and when this process ends, we will be left with an acyclic connected subgraph on $V(G)$, thereby a spanning tree. $\square$

2.6 Structural properties of trees

Theorem 2.25. The following are equivalent:

1. G is a tree.

2. $\forall u, v, u \neq v$, there is a unique uv -path.

3. G is maximally acyclic

4. G is minimally connected

(maximal and minimal are with respect to the edge set E under inclusion).

Proof. We will show $1 \Rightarrow 2 \Rightarrow 3 \Rightarrow 4 \Rightarrow 1$.

$1 \Rightarrow 2$: Since G is connected, there is a path between any two vertices. For contradiction, let there be two distinct paths between u and v . Consider the symmetric difference of the edges of the two paths (that is, consider only the edges which occur in exactly one of the paths). In the subgraph formed by these edges, the degree of every vertex (with respect to this subgraph) is even. Since the paths are distinct, the edge set is non-empty, and a non-empty graph where every vertex has even degree contains a cycle (why?).

$2 \Rightarrow 3$: A cycle gives us two paths between the vertices of the cycle, so assuming 2, it is clear that G is acyclic. Further let $(u, v) \notin E(G)$. There is a unique u, v -path in G , so adding the edge (u, v) will create a cycle, so G is maximally acyclic.

$3 \Rightarrow 4$: G must be connected, otherwise adding an edge between two vertices in different components would not create a cycle (as no path exists between them), contradicting the maximality of G . On the other hand, let $(u, v) \in E(G)$ such that removing this edge does not disconnect the graph. Then there is a u, v -path in G that does not use the edge (u, v) . This path along with the edge (u, v) gives us a cycle in G , which is a contradiction since G is acyclic.

$4 \Rightarrow 1$: If G has a cycle, then removing any one edge of the cycle will not disconnect the graph. So G must be acyclic, and hence a tree. $\square$

3 Lecture 3

BFS, Dijkstra's algorithm.

3.1 Breadth First Search

The breadth first search algorithm receives a graph G and a starting vertex s as input. The following are the important variables/lists given in the tables of examples and are maintained throughout the algorithm:

- j : index of the newly discovered vertex
- k : index of the current vertex being processed.
- $b(j)$: j th vertex discovered
- $d(v)$: distance from s . It can also be considered as the depth of a vertex. $d(s) = 0$ and initially $d(v) = \infty, \forall v \neq s$. (Note: If the graph is disconnected, $d(v) = \infty$ for any v unreachable from s . If we restart the BFS to find a forest, we reset $d(v) = 0$ for the new start vertex in the next component.)
- $p(v)$: parent of v (this is always empty for s and initially set empty for every other vertex. Note that if G is disconnected, there will be several vertices without a parent, exactly one for each component).

CAREFUL! Here $d(v)$ is not the degree while it is the exact same notation!

Input: $G = (V, E)$ and a vertex $s \in V$.

1. $j = 1; k = 1; b(1) = s$
2. $d(s) = 0$; for all other vertices $v \in V, v \neq s, d(v) = \infty$
3. **loop**
4. **if** $b(k)$ has a neighbor for which $d(v) = \infty$, **then:**

```

5.       $j = j + 1$ 
6.       $b(j) = v$ 
7.       $d(v) = d(b(k)) + 1$ 
8.       $p(v) = b(k)$ 
9.      otherwise:
10.     if  $k == j$ , then:
11.         stop
12.     otherwise:
13.          $k = k + 1$ 
14. end loop

```

We can use BFS to check if graph is disconnected. When the search cannot explore any new vertices, but there are unexplored vertices still, then the graph is disconnected. In this case, we restart the BFS from a new unexplored vertex v (with $d(v) = 0$ and $p(v)$ empty as for s) to get the BFS forest. The algorithm works just the same for directed graphs. The BFS also gives shortest distance in the directed version, only one has to be careful to note that $d(u, v)$ is not necessarily equal to $d(v, u)$ in this case.

Example 3.1. Consider the graph given below:

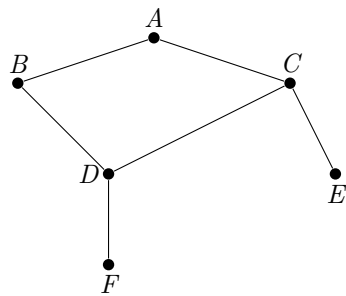
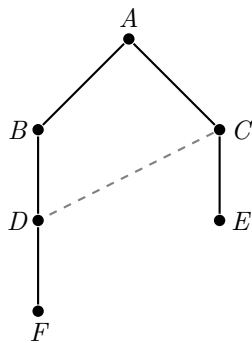


Table:

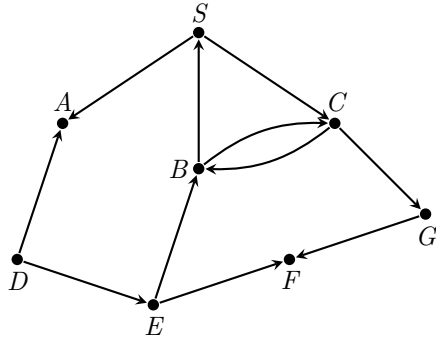
j (<i>Index</i>)	1	2	3	4	5	6
$b(j)$ (<i>Vertex</i>)	A	B	C	D	E	F
$d(v)$ (<i>Depth</i>)	0	1	1	2	2	3
$p(v)$ (<i>Parent</i>)	-	A	A	B	C	D

Note: When processing vertex C ($k = 3$), we check neighbor D . However, since $d(D) = 2$ (not ∞), D is already discovered and we do not update $b(j)$ or $d(D)$.

BFS Spanning Tree:



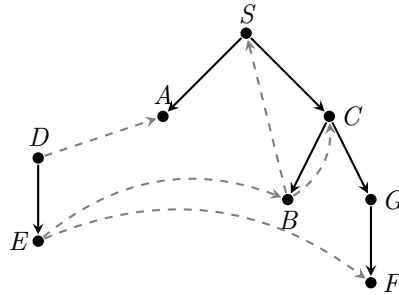
Example 3.2. Consider the directed graph given below:



Discovery and Distance Table:

j (Index)	1	2	3	4	5	6	7	8
$b(j)$ (Vertex)	S	A	C	B	G	F	D	E
$d(v)$ (Depth)	0	1	1	2	2	3	0	1
$p(v)$ (Parent)	-	S	S	C	C	G	-	D

BFS Spanning Forest:



Exercise 3.1. Find the BFS tree for the example above when starting at the vertex D.

Exercise 3.2. Is it true that if the undirected version of a directed graph (obtained by just removing the directions on each edge) is connected, then there is a BFS tree that is connected? Prove or give a counterexample.

Example 3.3. Consider the graph given below:

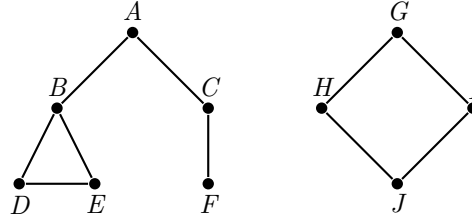
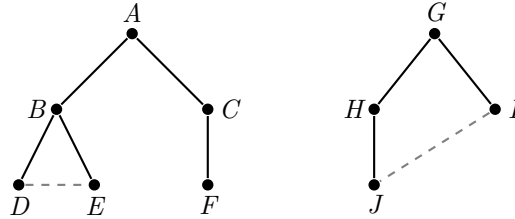


Table (Restarting for the second component):

j (Index)	1	2	3	4	5	6	7	8	9	10
$b(j)$ (Vertex)	A	B	C	D	E	F	G	H	I	J
$d(v)$ (Depth)	0	1	1	2	2	2	0	1	1	2
$p(v)$ (Parent)	-	A	A	B	B	C	-	G	G	H

Note: After vertex F is processed ($k = 6$), the queue is empty but $j < 10$. The algorithm restarts at G (the next unvisited vertex), resetting the depth $d(G) = 0$. No parent exists for G as it is a root of its own component.

BFS Spanning Forest (Undirected Example):



Theorem 3.4. The BFS algorithm takes at most $c \cdot (n + m)$ steps, where $n = |V(G)|$ and $m = |E(G)|$.

One simple observation that helps understand the theorems below is that the depth or distance assigned to vertices in a component is increasing. That is, if u is discovered/processed before v , then $d(u) \leq d(v)$. We now note the following two properties of the BFS tree for an undirected graph.

Theorem 3.5. For any edge $uv \in E(G)$, $|d(u) - d(v)| \leq 1$.

Proof. Without loss of generality, let u be processed before v and hence $d(u) \leq d(v)$. If v is undiscovered when u is being processed, then u is assigned as the parent of v and $d(v) = d(u) + 1$. If v was discovered earlier, then $d(v) \leq d(u) + 1$, since vertices expanded before u have depth at most $d(u)$ and v is a child of one such vertex. So $d(u) \leq d(v) \leq d(u) + 1$. $\square$

Theorem 3.6. The distance given by the BFS algorithm between s and any vertex v is the shortest distance between s and v , that is, there is no shorter path between them.

Proof. We note that recursively reaching to the parents of v , we will find a path between s and v in the BFS tree of length $d(v)$. Using the claim above, the distance $d(u)$ of intermediate vertices on any path from s to v can be at most 1 more than the previous vertex. So length of the path is $\geq d(v)$. $\square$

Exercise 3.3. Show that the above theorem also holds in the directed case. Note: you must be careful to modify the earlier theorem (for any edge $uv \in E(G)$, $|d(u) - d(v)| \leq 1$) appropriately.

3.2 Dijkstra's Algorithm

Given a weighted graph $G = (V, E)$ with weight function $w : E \rightarrow [0, \infty)$ and a vertex $s \in V$, we want to find minimum weight sv -paths for all vertices $v \in V, v \neq s$. (Here weight of a path is the sum of the weights of the edges in it).

Definition 3.7. Let $d(u, v)$ denote the smallest weight path between the vertices u and v .

The following are the important variables/lists given in the tables of examples and are maintained throughout the algorithm:

- $t(v)$: list of temporary distances of vertices v from s .
- $p(v)$: list of parents of vertices v from s .
- K : set of vertices that have been processed and their shortest distance will not change, that is, if $v \in K$, then $d(s, v) = t(v)$. The algorithm ends when $K = V$.

Input: $G = (V, E)$ a graph (can be directed), a vertex $s \in V$ and a non-negative weight function $w : E \rightarrow \mathbb{R}^+ \cup \{0\}$.

1. $t(s) = 0$; for all $v \in V, v \neq s$, if $(s, v) \in E(G)$, $t(v) = w((s, v))$, otherwise, $t(v) = \infty$
2. for all $v \in V, v \neq s$, if $(s, v) \in E(G)$, $p(v) = s$, otherwise, $p(v) = *$
3. $K = \{s\}$
4. **loop 1:** while $K \neq V$
 5. let v be a vertex in $V \setminus K$ with minimum $t(v)$ value.
 6. **loop 2:** for every edge $e = (v, u), u \notin K$
 7. **if** $t(u) > t(v) + w(e)$, then:
 8. $t(u) = t(v) + w(e)$
 9. $p(u) = v$
 10. **end loop 2**
 11. $K = K \cup \{v\}$
12. **end loop 1**

Here edges have been written as (u, v) . If Dijkstra's algorithm is being used on a directed graph, then this edge should be understood as $\vec{uv}$. The key thing to note about the algorithm is that for every vertex added to K , it updates the distances to all vertices with an incoming edge from this vertex.

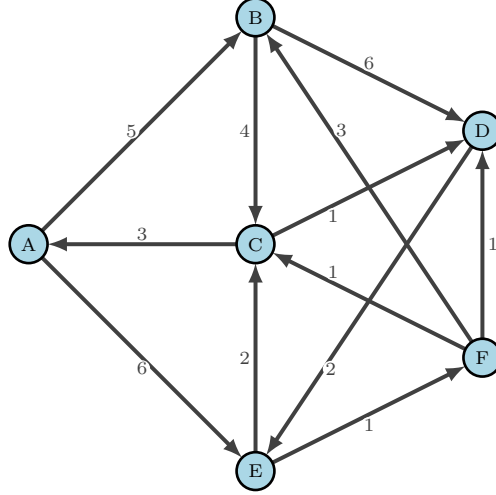
Theorem 3.8. Let $n = |V(G)|$ and $m = |E(G)|$. Then, the Dijkstra's algorithm takes $O(n^2)$ time with a simple array. With heaps, this can be improved to $O(m + n \log n)$.

Proof. Finding the minimum of the list $t(v)$ at every stage takes $c'n$ steps with a simple array (linear search) and since we repeat the cycle n times, we get the previously stated running time. We also note

that every edge in the graph is relaxed (updating the $t(v)$ value across an edge) at most once, and since number of edges is bounded by n^2 , so this running time takes into account that too.

We also note that with heaps, we can determine the minimum of $t(v)$ in $O(\log n)$ time and we find the minimum at most n times. Further the edges are relaxed at most once, so this gives us the tighter bound of $O(m + n \log n)$. $\square$

For an example, consider the following graph and the problem of finding shortest paths from A :



The following table shows the steps of Dijkstra's algorithm. The leftmost column shows the vertex $v \in V \setminus K$ with the current minimum $t(v)$ being moved into K . Also the colored distances show final distances which are not going to change since the vertices have been added to K .

K	$S = A$	B	C	D	E	F	$p(B)$	$p(C)$	$p(D)$	$p(E)$	$p(F)$
A	0	5	∞	∞	6	∞	A	*	*	A	*
B	0	5	9	11	6	∞	A	B	B	A	*
E	0	5	8	11	6	7	A	E	B	A	E
F	0	5	8	8	6	7	A	E	F	A	E

Since C and D both have $t(v)$ as 8, their shortest distance cannot change further. So we have stopped the algorithm here. But please note that the algorithm, as run by a computer, will only stop after every vertex is processed and $K = V$. This will add two more rows identical to the last row except in the first column where the current vertex added to K is being noted.

We need the following claim to prove the correctness of the Dijkstra's algorithm. Notice that we prove something stronger, namely that once a vertex is added to K , its shortest distance has been discovered, and for every vertex outside, $t(v)$ maintains the shortest distance using only vertices in K .

Theorem 3.9. *At every stage of the running of the Dijkstra's algorithm, the following two statements are true: (Note, we call this the **Invariant** of the algorithm)*

1. $\forall u \in K, t(u) = d(s, u)$
2. $\forall u \notin K, t(u)$ is the shortest distance of a su -path in the subgraph induced on the vertices $K \cup \{u\}$.

Proof. We will use the notation that for any two vertices, if uv is an edge in the graph, then $w(uv)$ is the weight of the edge as defined by the weight function, but if there is no such edge in the graph, then we define $w(uv) = \infty$.

We will use induction on the size of K . We initialize with $K = \{s\}$, then $d(s, s) = t(s) = 0$ and for $u \neq s, t(u) = w(su)$. These values satisfy both the above conditions.

Let $K \neq V(G)$ and let the claim be true for K . Now, we suppose that the algorithm picks a new vertex $v \notin K$ and we add it to K . Then, we need to check that the claim still holds.

For the first statement, we need to check that $t(v) = d(s, v)$. For this, we note that $t(u)$ is (by induction) the shortest path to any vertex using only vertices in $K \cup \{u\}$. And v is the vertex with the smallest $t(v)$ value from the vertices in $V(G) \setminus K$. Since any path to v must exit K , the shortest sv -path must directly exit K to v , since if it exits K to another vertex u , then the path to v will be longer. This is because $t(u)$ is the shortest path using only vertices of K and u (Induction hypothesis) and $t(v)$ is the minimum value of all vertices in $V(G) \setminus K$. So $t(v) = d(s, v)$. (Note: it is critical to this proof that the edges have non-negative weight).

For seeing that the second statement also holds, we need to check that on the graph induced by $K \cup \{u, v\}$, for any vertex $u \notin K$ and $u \neq v$, the updated $t(u)$ is the shortest path. We notice that the way we update $t(u)$ is, $t(u) = \min(t(u), t(v) + w(vu))$. Since the only new path in this induced subgraph can be through v and we are updating t as the shorter of these two paths, we are making sure that $t(u)$ remains the shortest path in $K \cup \{u, v\}$. $\square$

From the claim we can see that when the algorithm ends with $K = V(G)$, we will have found the shortest distances to all vertices.

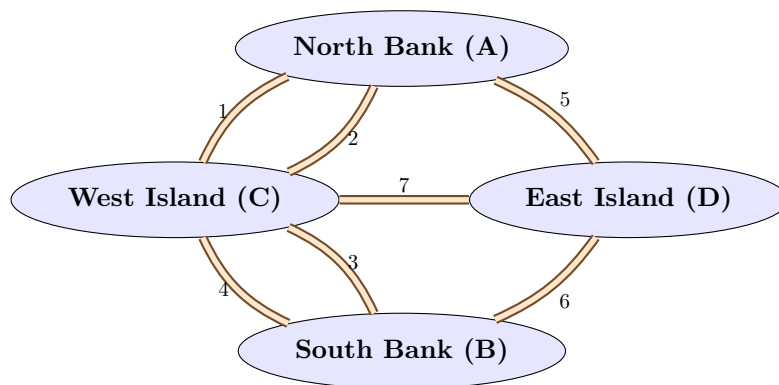
Exercise 3.4. Use the Dijkstra's algorithm to give a new proof that the BFS algorithm returns the shortest path between s and other vertices in an unweighted graph.

4 Lecture 4

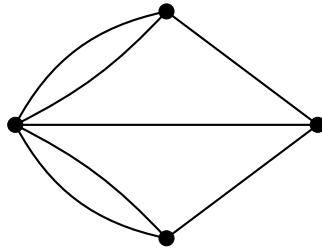
Eulerian circuits, Hamiltonian cycles.

4.1 The birth of graph theory

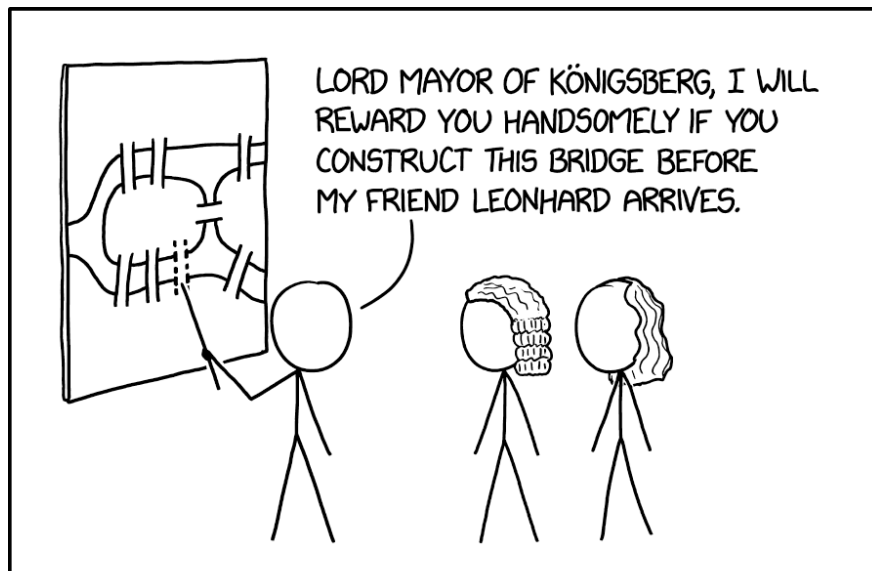
In the 18th century, the city of Königsberg, Prussia (now Kaliningrad, Russia), was set on both sides of the Pregel River. The river contained two large islands, and the four landmasses were connected by seven bridges.



A popular local pastime involved a riddle: *Is it possible to take a walk through the city crossing each of the seven bridges exactly once and returning to the starting point?* The riddle was introduced to the Swiss mathematician Leonhard Euler during his visit to the city. Euler realized that standard geometry (which deals with lengths and angles) was not required for analyzing the problem. He realized the size of the islands and the length of the bridges didn't matter. He treated landmasses as points (vertices) and bridges as lines (edges). And represented as a graph, he could prove, decisively that it was impossible to walk along the bridges of Königsberg exactly once.



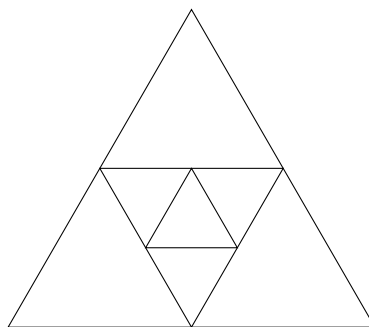
As some of you might have noticed, the thumbnail of our Teams group is a comic. Its the Xkcd comic that I really enjoy reading. At this very relevant point, I would introduce the thumbnail of our Teams group as a larger picture.



I TRIED TO USE A TIME MACHINE TO CHEAT ON MY ALGORITHMS FINAL BY PREVENTING GRAPH THEORY FROM BEING INVENTED.

4.2 Eulerian circuits

Can you draw the following without lifting the pencil:



Definition 4.1. *Trivial components* are components of the graph that have no edges. These are also called *isolated vertices*.

Definition 4.2. An *Eulerian circuit* in a graph G is a closed trail (also called a circuit) that goes through every edge of G exactly once.

Definition 4.3. An *Eulerian trail* in a graph G is a trail that goes through every edge of G exactly once. Note: it only differ from the previous definition in that it needn't start and end at the same vertex.

Definition 4.4. A graph is said to be an **even graph** if the degree of every vertex is even.

The following theorem was done in Lecture 2 as an exercise.

Theorem 4.5. Every non-trivial even graph contains a cycle.

Proof. If the graph has a loop, then the loop is a cycle. Otherwise, consider a maximal path in the graph. Let u be the first vertex of the path. Since $d(u) > 1$ and this is a maximal path, it must have another neighbor on the path. This will give us the required cycle. $\square$

Theorem 4.6. A graph G has an Eulerian circuit if and only if G is even and is connected.

Proof. If a graph has an Eulerian circuit, then as we walk on this circuit, suppose the edge (u, v) is traversed from u to v . Then direct the edge (u, v) as $\vec{uv}$. This way, we will get a digraph. The Eulerian circuit tells us that $d^+(v) = d^-(v)$ for every vertex. This would imply that $d(v)$ was even in the original graph.

Now we must show the converse, that is, if the graph is even with one non trivial component, then it has an Eulerian circuit. We will give two proofs for this. The first proof uses Induction, while in the second we give an algorithm.

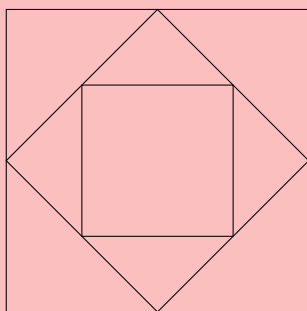
Inductive proof: We will use induction on the number of edges in the graph. If $m = |E(G)| = 0$, then the graph trivially has an Eulerian circuit. For $m > 0$, then by the lemma above, we can find a cycle C in G . Removing the edges of this cycle may disconnect the graph and there might be more than one component, but each component will in turn be a connected even graph and with induction hypothesis will contain an Eulerian circuit. Also, since the original graph was connected, all these components must share a vertex with the cycle. Then to get the Eulerian circuit for G , we walk along the cycle C but whenever we reach a vertex belonging to some component, we walk along its Eulerian circuit before returning to the cycle.

Sketch of the algorithm: We can also see this proof as an algorithm. Pick any starting vertex v . We start a walk here at v making sure we walk on every edge exactly once. Since the degree of every vertex is even, the only time we cannot walk further will be if we reach back v and we have walked on all edges incident onto v . If all the edges have been walked on, then this was the required Eulerian circuit. If not, we can pick any vertex, all whose edges have not been walked on, and start this process again. In the end, we will join all these walks to get the required Eulerian circuit. $\square$

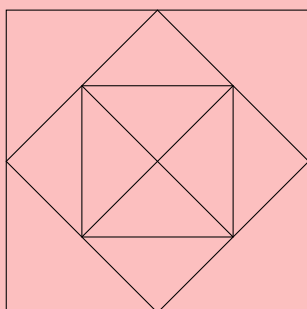
Theorem 4.7. A graph G has an Eulerian trail if and only if it has only one non trivial component and exactly two vertices of odd degree.

Proof. If a graph has an Eulerian trail, we again create a digraph as above and the result follows. For the converse, we connect the two odd degree vertices with an edge to get an even graph. This will have an Eulerian circuit. Deleting this extra edge will give us the required Eulerian trail. $\square$

Exercise 4.1. Can you draw the following without lifting the pencil?

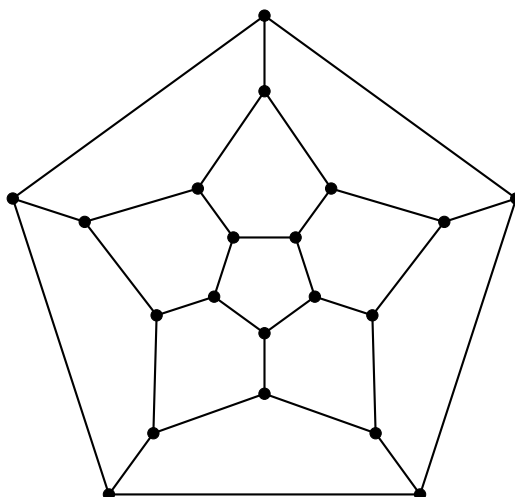


Exercise 4.2. *Can you draw the following without lifting the pencil?*



4.3 Hamiltonian paths, cycles

The "Traveller's Dodecahedron" game (officially marketed as The Icosian Game, invented by the famous Irish mathematician Sir William Rowan Hamilton in 1857) is essentially the sequel to the Seven Bridges of Königsberg in the history of graph theory. The game consisted of a wooden board representing the skeleton of a dodecahedron. The 20 vertices of the dodecahedron were labeled with the names of famous world cities (e.g., London, Delhi, Paris). A traveller had to find a route along the edges that visited every city exactly once and returned to the starting point. Although the game was a commercial flop, its impact on mathematics and computer science is with few parallels, with determining if a graph has such a cycle being a very important problem in the class of NP-complete problems.



Definition 4.8. A **Hamiltonian cycle** in a graph G is a cycle that goes through every vertex of the graph. It is also called a *spanning cycle*.

Definition 4.9. A **Hamiltonian path** in a graph G is a path that goes through every vertex of the graph.

Definition 4.10. A graph is said to be **Hamiltonian** if it contains a Hamiltonian cycle.

Theorem 4.11. (Necessary condition for containing a Hamiltonian cycle): If a graph G is Hamiltonian, then $\forall S \subset V(G)$, the graph $G \setminus S$ has at most $|S|$ components.

Proof. Let C be the Hamiltonian cycle in G . Deleting the vertices in S creates some components in G . The Hamiltonian cycle C cannot go between these components and must go from any component to S before travelling to another component. Let us mark all the arrival vertices into S , that is, every time the cycle goes from a vertex in a component in $G \setminus S$ to a vertex in S , we will mark this vertex. We notice that there must be at least as many distinct arrival vertices as components in $G \setminus S$. $\square$

Theorem 4.12. (Necessary condition for containing a Hamiltonian path): If a graph G contains a Hamiltonian path, then $\forall S \subset V(G)$, the graph $G \setminus S$ has at most $|S| + 1$ components.

Proof. The same proof as for the previous statement, only here at most one component need not have an arrival vertex. $\square$

Exercise 4.3. Consider the 3×3 chessboard where two squares are adjacent if they share an edge. Is the graph Hamiltonian? Does it have a Hamiltonian path?

Exercise 4.4. Consider the cube made by stacking 27 smaller cubes of unit length to obtain a $3 \times 3 \times 3$ cube. From this we now remove the center cube. Two smaller cubes are adjacent if they share a face. Is the graph Hamiltonian? Does it have a Hamiltonian path?

Are there any graphs which satisfy the necessary condition given above but are not Hamiltonian? Graphs which demonstrate that the above condition is not sufficient?

Exercise 4.5. Show that the Peterson graph satisfies the necessary condition for being Hamiltonian. Further show that Peterson graph has no Hamiltonian cycle.

Examples of non-Hamiltonian graphs which satisfy the necessary condition are a little harder to find. The Goldner-Harary graph is another such example.

Definition 4.13. The minimum degree of a graph, denoted by $\delta(G)$ or simply δ is the minimum of all degrees in the graph.

Definition 4.14. The maximum degree of a graph, denoted by $\Delta(G)$ or simply Δ is the maximum of all degrees in the graph.

Theorem 4.15. (Dirac's): (sufficient condition for containing a Hamiltonian cycle): if G is a graph such that $|V(G)| \geq 3$, and $\delta(G) \geq n/2$, then G is Hamiltonian.

Proof. For a proof by contradiction, assume that G satisfies the conditions mentioned in the theorem, but does not contain a Hamiltonian cycle. Let $(u, v) \notin E(G)$. If after adding the edge (u, v) , the graph still does not contain a Hamiltonian cycle, then add it. We repeat this process until adding any further edges will make the graph contain a Hamiltonian cycle. Let this saturated graph be H . We notice that H also satisfies the conditions in the theorem. We will also note that H must contain a Hamiltonian path, otherwise we could have added more edges while not creating a Hamiltonian cycle. Let the vertices on the Hamiltonian path be $u_1, u_2, \dots, u_n$. Let $I = \{i | (u_1, u_i) \in E(H)\}$ and let $J = \{i + 1 | (u_i, u_n) \in E(H)\}$. Since H is not Hamiltonian, u_1 and u_n are not adjacent. So indices in I and J are between 2 and n , a total of $n - 1$ indices, while $d(u_1) + d(u_n) \geq n$, so $I \cap J \neq \emptyset$. From this we can construct the Hamiltonian cycle. This contradiction will imply that our original assumption, that G is not Hamiltonian, is false. $\square$

Theorem 4.16. (*Ore's*): (sufficient condition for containing a Hamiltonian cycle): if G is a graph such that $|V(G)| \geq 3$, and for any two non-adjacent vertices u and v , $d(u) + d(v) \geq n$, then G is Hamiltonian.

Proof is exactly the same as the proof for Dirac's, and so is omitted.

A cycle graph (graph that is a cycle) has a Hamiltonian cycle, but does not satisfy degree requirements of Dirac's or Ore's theorem. So this is an example of when the conditions of Dirac's or Ore's theorem are not met, but the graph is still Hamiltonian. In other words, the conditions in those theorems are sufficient for containing a Hamiltonian cycle, but not necessary.